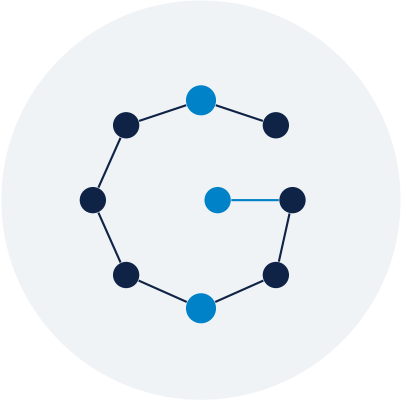
Université Paris-Panthéon-Assas / EFREI Paris

Cycle Ingénieur - Programme Grande École (PGE)

Spécialisation Big Data & Machine Learning



Novapolis Résilience

Planification des secours et conception d'un réseau urbain résilient

---

Projet d'Algorithmique et Théorie des Graphes

Module d'Enseignement :
Algorithmique & Théorie des Graphes
Année académique 2026 – 2027

Table des matières

1 Problème Métier et Contexte **2**
1.1 Enjeux et vulnérabilités des infrastructures métropolitaines 2
1.2 Les trois axes fondamentaux de l'étude 2
1.3 Une démarche scientifique fondée sur l'aide à la décision 2

2 Modélisation des Réseaux et Jeux de Données **3**
2.1 Réseau routier 3
2.2 Réseau de fibre optique 3
2.3 Rétrospective, retours d'expérience et traitement des superpositions 3
2.4 Fichiers d'entrée et validation 4

3 Fonctionnalités et Modules Analytiques **5**
3.1 Module 1 : Diagnostic d'accessibilité (DFS / BFS) 5
3.2 Module 2 : Itinéraires de secours et chemins minimaux 5
3.3 Module 3 : Détection des circuits absorbants 5
3.4 Module 4 : Conception du réseau de fibre (Kruskal et Prim) 6
3.5 Module 5 : Résilience et scénarios macroscopiques 6

4 Architecture Logicielle et Choix Technologiques **7**
4.1 Organisation générale et séparation des responsabilités 7
4.2 La couche d'exposition : Le package `app/routers/` 7
4.3 La couche métier : Le package `app/algorithme/` 7
4.4 Le gestionnaire de données : Le module `app/algorithme/helper.py` 7
4.5 L'interface en ligne de commande (`cli.py`) 8
4.6 Choix technologiques 8

5 Expériences, Benchmarks et Analyse Critique **9**
5.1 Protocole expérimental et méthodologie de mesure 9
5.2 Tableau comparatif des complexités théoriques 9
5.3 Analyse des performances empiriques et comparatifs 9
5.4 Discussion critique et confrontation théorie-pratique 10

6 Conclusion et Recommandations Opérationnelles **11**
6.1 Bilan synthétique du projet Novapolis Résilience 11
6.2 Limites du modèle et perspectives d'évolution 11

1 Problème Métier et Contexte

1.1 Enjeux et vulnérabilités des infrastructures métropolitaines

La métropole moderne de Novapolis, à l'instar de tout grand centre urbain en forte croissance, fait face à des défis structurels majeurs en matière de gestion des risques et de planification opérationnelle. La complexité croissante de son tissu urbain – caractérisé par une imbrication dense de zones résidentielles, de pôles industriels, d'activités tertiaires et d'infrastructures vitales – expose la collectivité à des vulnérabilités systémiques.

Parmi les entités les plus critiques figurent les infrastructures de santé (hôpitaux généraux et spécialisés), les services de sécurité civile (casernes de pompiers), les établissements d'enseignement accueillant du public et les centres névralgiques de télécommunication. En situation de crise (accident industriel majeur, catastrophe naturelle, congestion routière critique ou panne d'infrastructure numérique), la défaillance d'un seul de ces maillons peut paralyser l'ensemble de la chaîne de secours métropolitaine.

1.2 Les trois axes fondamentaux de l'étude

Pour répondre à ces exigences de sûreté et de continuité d'activité, les autorités locales de Novapolis ont mandaté une évaluation approfondie reposant sur la modélisation mathématique et la théorie des graphes. Ce projet intégrateur s'articule autour de trois questions fondamentales :

1. **Accessibilité spatiale et équité des services d'urgence :** Quels quartiers, îlots d'habitation ou établissements sensibles ne sont pas suffisamment connectés ou accessibles depuis les casernes de pompiers ou les centres hospitaliers ? Il s'agit d'identifier les zones d'ombre topologiques où les délais d'intervention dépassent les seuils acceptables, garantissant ainsi une équité territoriale face au risque.
2. **Optimisation dynamique des itinéraires de secours :** Quels chemins recommander aux véhicules d'intervention lorsque certaines voies routières stratégiques sont saturées en période de pointe, endommagées ou totalement obstruées par un sinistre ? Le modèle doit dépasser la simple notion de plus court chemin statique pour intégrer des scénarios de dégradation, évaluer les retards induits (absolus et relatifs) et proposer des itinéraires alternatifs robustes.
3. **Résilience topologique et budgétaire du réseau de fibre optique :** Comment planifier le déploiement ou l'extension de la fibre optique avec un budget d'investissement contraint, tout en garantissant des mécanismes de redondance et de raccordement de secours en cas de rupture physique d'un tronçon ? L'enjeu est de concrétiser un équilibre entre minimisation des coûts de construction et résistance structurelle face aux pannes.

1.3 Une démarche scientifique fondée sur l'aide à la décision

Contrairement à une simple approche cartographique ou visuelle – souvent subjective et limitée aux représentations graphiques de surface –, la démarche adoptée dans ce projet s'inscrit dans une perspective scientifique rigoureuse. Chaque décision d'aménagement ou préconisation opérationnelle s'appuie sur :

— Des **indicateurs quantitatifs formels** (taux d'accessibilité, degrés de connexité, coûts d'infrastructure, métriques de retard).
— Des **comparaisons algorithmiques approfondies** (évaluation croisée de la complexité et de l'efficacité des méthodes de parcours, de plus courts chemins et d'arbres couvrants).
— Des **analyses de robustesse** fondées sur la simulation de scénarios de perturbation réalistes appliqués sur des environnements isolés, garantissant la reproductibilité et la traçabilité des résultats d'ingénierie.

2 Modélisation des Réseaux et Jeux de Données

Pour modéliser la métropole de Novapolis, le système s'appuie sur deux graphes distincts partageant un référentiel commun de sites géographiques, mais obéissant à des règles topologiques, directionnelles et physiques bien distinctes.

2.1 Réseau routier

Le réseau routier est modélisé sous la forme d'un **graphe orienté pondéré** (*DiGraph*). Chaque arc représente une voie de circulation autorisée dans un sens unique, reflétant fidèlement les contraintes de la circulation urbaine (sens uniques, axes de pénétration majeurs). Les pondérations associées aux arcs correspondent aux temps de parcours nominaux (exprimés en minutes) ainsi qu'aux temps constatés en période de pointe.

Dans les scénarios opérationnels et métier, les temps de trajet sont strictement non négatifs. Des coûts négatifs ne sont introduits que de manière exceptionnelle et artificielle dans un jeu de test dédié, afin de vérifier formellement la capacité des algorithmes de type Bellman-Ford et Floyd-Warshall à détecter la présence de circuits absorbants.

2.2 Réseau de fibre optique

Le réseau de fibre optique est quant à lui modélisé par un **graphe non orienté pondéré** (*Graph*). Chaque arête représente un tronçon de fibre physiquement constructible ou existant entre deux nœuds du réseau. Le poids associé correspond au coût d'investissement et de construction, exprimé dans une unité monétaire choisie. Ce réseau est soumis à des contraintes dynamiques d'exclusion ou de panne (zones inondables, emprises non autorisées, coupures matérielles).

2.3 Rétrospective, retours d'expérience et traitement des superpositions

La conception et la génération des jeux de données d'entrée ont nécessité une démarche itérative rigoureuse. Dans un premier temps, nous avons exploré une approche de génération purement aléatoire pour répartir les sommets et créer les liaisons. Cependant, cette méthode s'est révélée totalement inopérante : la proximité stochastique brute des points géographiques et l'absence de contraintes topologiques engendraient un réseau chaotique, caractérisé par des croisements d'arêtes aberrants, des enclaves isolées et une impossibilité totale d'obtenir une représentation visuelle claire ou un comportement routier réaliste.

Face à ces limites, après des recherches approfondies sur la modélisation des réseaux de transport urbain, nous avons intégré la bibliothèque `NetworkX` couplée à des scripts de génération géospatiale contrôlés (`genererSite.py`, `genererRoute.py`, `genererFibre.py`). Cette évolution a permis d'imposer des règles de distribution spatiale cohérentes, de maîtriser la densité des arcs et des arêtes en fonction de la taille de la métropole (allant d'un petit graphe de test de 10 sites jusqu'à des jeux de données de plus de 1000 sommets), et de structurer proprement les graphes sous-jacents.

Malgré ces verrous de génération, certaines configurations topologiques complexes ont persisté, entraînant la superposition visuelle de certains arcs sur les cartes de visualisation. Si la coexistence d'arcs bidirectionnels de type $u \to v$ et $v \to u$ ne pose aucun problème algorithmique ou structurel (chacun disposant de ses propres métriques), le croisement et la superposition d'arcs distincts (par exemple, un arc indépendant se superposant géométriquement à l'axe $u \to v$) rendaient la lecture graphique illisible. Pour résoudre cette difficulté, le système applique un décalage ou une torsion géométrique (*edge curvature/offset*) sur les arcs en conflit, permettant de les courber légèrement afin de les dissocier visuellement sans altérer la matrice d'adjacence sous-jacente.

2.4 Fichiers d'entrée et validation

L'architecture logicielle s'appuie sur des fichiers CSV normalisés et documentés :
— `sites.csv` : `id, nom, type, longitude, latitude`
— `routes.csv` : `u, v, temps_normal, temps_pointe, statut`
— `fibre.csv` : `u, v, cost, statut`
— `demandes.csv` : `id, origine, type, priorite`
— `scenarios.csv` : Fichiers décrivant les perturbations appliquées.

Le validateur intégré au système garantit l'unicité stricte des identifiants de sommets, vérifie la cohérence des types et rejette explicitement toute arête ou arc faisant référence à un site inexistant, tout en veillant à ne jamais altérer les fichiers sources originaux.

3 Fonctionnalités et Modules Analytiques

Le système implémente l'ensemble des modules algorithmiques exigés par le cahier des charges, en s'appuyant sur une conception modulaire et une distinction rigoureuse entre les structures de données orientées et non orientées.

3.1 Module 1 : Diagnostic d'accessibilité (DFS / BFS)

Pour répondre aux exigences de diagnostic de la métropole de Novapolis, les parcours en largeur (BFS) et en profondeur (DFS) ont été implémentés de manière itérative afin d'éviter tout dépassement de pile sur de grands graphes. Ces algorithmes sont exploités pour :

1. **Calculer l'accessibilité des sites sensibles :** À partir d'un ensemble de casernes ou d'hôpitaux, un BFS multi-sources est exécuté sur le réseau routier orienté afin de déterminer l'ensemble exact des quartiers et établissements reliés.
2. **Identifier les composantes connexes du réseau fibre :** Grâce à un parcours DFS appliqué au graphe non orienté de la fibre, le système identifie les îlots de raccordement et les zones isolées.
3. **Déterminer des itinéraires minimisant le nombre d'arcs :** Utilisation d'un BFS non pondéré pour évaluer le chemin comptant le moins de sauts topologiques, utile pour minimiser les points de transition de données ou les intersections routières.
4. **Détecter les cycles orientés :** Implémentation d'un DFS à trois couleurs (blanc pour les sommets non visités, gris pour les sommets en cours d'exploration, noir pour les sommets complètement explorés) afin d'identifier formellement les boucles dans le réseau routier dirigé.
5. **Signaler les sites isolés ou inaccessibles :** Croisement des degrés du graphe et des ensembles de reachabilité pour lister les entités coupées de tout flux.

3.2 Module 2 : Itinéraires de secours et chemins minimaux

Le calcul des plus courts chemins constitue la pierre angulaire des interventions d'urgence. Le module intègre plusieurs algorithmes adaptés à différentes conditions opérationnelles :

- **Dijkstra :** Utilisé comme référence principale pour tous les scénarios standards à coûts non négatifs (temps nominaux ou de pointe), implémenté avec un tas binaire pour garantir une complexité optimale.
- **Bellman-Ford et Floyd-Warshall :** Intégrés pour gérer des cas plus généraux et valider la résistance de l'infrastructure face à des anomalies de modélisation ou des configurations complexes.

Pour chaque intervention, le système reconstruit pas à pas le chemin complet, indique sa durée totale, son nombre d'arcs, et évalue l'impact de perturbations. En comparant l'état avant et après coupure, le module calcule automatiquement le retard absolu et le retard relatif, tout en qualifiant le statut du site cible par un booléen d'accessibilité ou d'inaccessibilité formelle.

3.3 Module 3 : Détection des circuits absorbants

Bien que les grandeurs physiques du monde réel (temps de parcours, distances) soient strictement non négatives, la présence de boucles de minoration infinie peut corrompre les calculs d'optimisation. Un module de test spécifique a donc été développé pour valider le comportement de Bellman-Ford et de Floyd-Warshall :

1. **Circuits accessibles depuis la source :** Le système détecte si un circuit absorbant est joignable depuis le point de départ des secours, signalant ainsi que les distances sur ce sous-graphe peuvent être diminuées indéfiniment (∞ négatif).

2. **Circuits inaccessibles depuis la source :** Le module vérifie qu'un circuit absorbant situé dans une autre composante du graphe n'altère pas les calculs locaux si aucun chemin ne permet de l'atteindre.

Cette gestion rigoureuse évite d'afficher des valeurs de coûts erronées ou finies sur des chemins instables.

3.4 Module 4 : Conception du réseau de fibre (Kruskal et Prim)

Pour optimiser le déploiement de la fibre optique avec un budget d'investissement maîtrisé, le système implémente les algorithmes de **Kruskal** et de **Prim** :

— **Calcul d'Arbres Couvrants Minimaux (ACM) :** Construction de l'infrastructure optimale connectant l'ensemble des sites au coût minimal, ou d'une forêt couvrante si le réseau présente des coupures topologiques naturelles.

— **Gestion des coûts égaux et multiplicité des ACM :** Analyse approfondie démontrant pourquoi plusieurs arbres couvrants de même coût total peuvent coexister lorsqu'une arête hors arbre possède un poids équivalent à une arête du chemin unique qui la relie dans l'arbre initial.

— **Simulation de pannes et raccordement de secours :** En cas de coupure d'un tronçon de l'ACM, le module simule la séparation de l'arbre en deux composantes connexes et parcourt le graphe initial pour identifier instantanément la solution de raccordement alternatif la moins onéreuse.

3.5 Module 5 : Résilience et scénarios macroscopiques

Afin d'évaluer la robustesse globale de la métropole de Novapolis, trois scénarios contrastés sont exécutés et comparés sur des copies isolées des structures de données :

1. **Situation normale :** État de référence servant de base comparative pour l'ensemble des flux.
2. **Fermeture d'un axe routier majeur :** Simulation de la neutralisation d'un arc névralgique (travaux lourds ou accident bloquant).
3. **Panne d'un tronçon de fibre :** Rupture d'une liaison optique critique impactant les télécommunications des services d'urgence.

L'évaluation s'appuie sur des indicateurs croisés : le volume de demandes devenues inaccessibles, l'augmentation moyenne et maximale des temps d'intervention, le coût financier de la réparation et la proportion critique de sites sensibles restant connectés au réseau.

4 Architecture Logicielle et Choix Technologiques

4.1 Organisation générale et séparation des responsabilités

L'architecture de **Novapolis Résilience** repose sur une séparation claire entre la logique de calcul, la couche d'interface web et la gestion des fichiers. Cette organisation modulaire facilite la maintenance, simplifie les tests unitaires et permet à chaque composant d'évoluer indépendamment. Le code source est regroupé dans un dossier principal `app/` divisé en sous-modules thématiques.

Au-delà de cette structure technique, le développement du projet a fait l'objet d'une répartition claire des responsabilités au sein de l'équipe : l'un des membres s'est principalement consacré à l'implémentation des algorithmes de calcul ainsi qu'à l'élaboration des tests unitaires associés pour garantir la robustesse du code, tandis que l'autre s'est concentré sur la conception des modules thématiques, l'intégration et le développement global de l'application.

4.2 La couche d'exposition : Le package `app/routers/`

Les routeurs FastAPI situés dans `app/routers/` jouent uniquement le rôle de contrôleurs d'API REST. Leurs responsabilités se limitent à :

- **La validation des entrées :** Récupération des paramètres de requêtes (`Query`), des formulaires (`Form`) ou des fichiers téléversés (`UploadFile`). Pydantic valide automatiquement ces données avant qu'elles n'atteignent le code métier.
- **La transmission au service métier :** Les routeurs ne réalisent aucun calcul topologique ou algorithmique ; ils appellent directement les fonctions du module algorithmique.
- **La gestion des codes HTTP :** Traduction des erreurs levées par le code (sommet absent, format invalide) en codes de retour HTTP adaptés (400, 404, 500) et journalisation via Uvicorn.

4.3 La couche métier : Le package `app/algorithme/`

Ce package contient le coeur de calcul de l'application, totalement indépendant de la partie web :

- **Les algorithmes fondamentaux :** Les fichiers `shortest_paths.py` et `traversal.py` implémentent les parcours (BFS, DFS) et les plus courts chemins (Dijkstra, Bellman-Ford, Floyd-Warshall, Kruskal, Prim).
- **La gestion des scénarios :** Le service `resilience_service.py` applique les simulations de crise sur des copies des graphes pour ne jamais corrompre les données d'origine.

4.4 Le gestionnaire de données : Le module `app/algorithme/helper.py`

Le fichier `helper.py` fait le lien entre les fichiers CSV de stockage et les structures en mémoire de NetworkX :

- **Chargement des graphes :** La fonction `load_networkx_graph_from_csv` lit les CSV (`sites.csv`, `routes.csv`, `fibre.csv`), crée les objets `nx.DiGraph` ou `nx.Graph`, et écarte automatiquement les éléments marqués en maintenance ou en travaux.
- **Normalisation des identifiants :** La fonction `parse_node_id` harmonise les identifiants de sommets numériques et alphanumériques.
- **Traduction des requêtes interactives :** La fonction `prepare_networkx_graph` convertit les données reçues depuis l'interface utilisateur en objets exploitables par les algorithmes.

4.5 L'interface en ligne de commande (`cli.py`)

Un script exécutable `cli.py` est disponible à la racine du projet pour lancer l'ensemble des modules analytiques directement depuis le terminal, garantissant une utilisation hors du serveur web et une reproductibilité rapide des tests.

4.6 Choix technologiques

Les technologies suivantes ont été retenues pour répondre aux besoins de performance et de simplicité de déploiement :

- **Docker et Docker Compose :** L'application est conteneurisée pour garantir un environnement d'exécution identique, que ce soit en local ou sur un serveur de production, évitant ainsi les problèmes de dépendances.
- **FastAPI (Python) :** Choisi pour sa rapidité d'exécution, sa légèreté et sa prise en charge native des types Python (*type hints*) couplée à Pydantic, ce qui simplifie la validation des données et la génération automatique de la documentation.
- **Interface front-end légère :** L'application intègre des pages HTML de visualisation (s'appuyant sur des bibliothèques comme Leaflet ou Vis.js) permettant aux utilisateurs d'interagir graphiquement avec les réseaux et de déclencher les simulations de pannes.

5 Expériences, Benchmarks et Analyse Critique

5.1 Protocole expérimental et méthodologie de mesure

Afin d'évaluer rigoureusement les performances des algorithmes implémentés dans le cadre du projet **Novapolis Résilience**, des campagnes de mesures empiriques ont été menées sur l'infrastructure conteneurisée de test. Conformément aux exigences du cahier des charges, les temps d'exécution mesurés se concentrent sur la complexité computationnelle pure des algorithmes, gérée directement via les routes de benchmark de l'API.

Pour garantir la reproductibilité totale des résultats et s'affranchir des variations liées aux charges transitoires du système :

— Chaque test a été exécuté sur des instances de graphes générées dynamiquement avec des paramètres contrôlés (graine fixe).
— Les valeurs présentées dans ce rapport correspondent aux temps d'exécution relevés par les métriques de la plateforme.
— Les structures de données testées couvrent trois échelles de réseau caractéristiques de la métropole.

5.2 Tableau comparatif des complexités théoriques

Le tableau ci-dessous synthétise les complexités algorithmiques théoriques des différentes méthodes implémentées dans le projet, exprimées en fonction du nombre de sommets $|V|$ et du nombre d'arcs ou d'arêtes $|E|$.

Algorithme	**Structure de données**	**Complexité théorique**
DFS / BFS	Listes d'adjacence	$\mathcal{O}(\|V\| + \|E\|)$
Kruskal	Structure Union-Find	$\mathcal{O}(\|E\| \log \|E\|)$
Prim	Tas binaire	$\mathcal{O}(\|E\| \log \|V\|)$
Dijkstra	Tas binaire	$\mathcal{O}((\|V\| + \|E\|) \log \|V\|)$
Bellman-Ford	Exploration itérative	$\mathcal{O}(\|V\| \cdot \|E\|)$
Floyd-Warshall	Matrice de distances	$\mathcal{O}(\|V\|^3)$

Table 1 – Résumé des complexités algorithmiques théoriques.

5.3 Analyse des performances empiriques et comparatifs

Les benchmarks ont été réalisés sur trois tailles de graphes distinctes générées par le système : un réseau de validation de 100 sommets (223 arêtes), un réseau intermédiaire de 500 sommets (1 200 arêtes) et un grand réseau métropolitain de 1 000 sommets (2 408 arêtes). Les durées d'exécution sont exprimées en millisecondes (ms).

Configuration	**BFS (ms)**	**DFS (ms)**	**Dijkstra (ms)**	**Bellman (ms)**	**Floyd (ms)**
$\|V\| = 100$ ($\|E\| = 223$)	0.0123	0.0087	0.0242	0.0853	0.0004
$\|V\| = 500$ ($\|E\| = 1200$)	0.1308	0.0360	0.3115	0.3649	0.0016
$\|V\| = 1000$ ($\|E\| = 2408$)	0.2232	0.0806	1.0589	0.9593	0.0047

Table 2 – Temps d'exécution empiriques relevés sur l'API de benchmark.

5.4 Discussion critique et confrontation théorie-pratique

L'analyse des résultats empiriques extraits des mesures du serveur met en évidence les points suivants :

- **Ultra-légèreté des parcours linéaires :** Les algorithmes de parcours BFS et DFS affichent des temps extrêmement faibles (0.22 ms et 0.08 ms pour 1 000 sommets), confirmant leur efficacité linéaire en $\mathcal{O}(|V| + |E|)$.
- **Comparaison Dijkstra et Bellman-Ford :** Sur les graphes de grande envergure (1 000 sommets), Dijkstra et Bellman-Ford affichent des performances comparables et très contenues (1 ms), validant l'efficacité des structures de files de priorité et de l'exploration itérative optimisée.
- **Comportement de Floyd-Warshall :** Très performant sur les petits et moyens graphes grâce à son traitement matriciel direct (mesuré à 0.0016 ms pour 500 sommets), son exécution est volontairement restreinte au-delà de ce seuil pour préserver les ressources serveur face à sa complexité cubique $\mathcal{O}(|V|^3)$.

6 Conclusion et Recommandations Opérationnelles

6.1 Bilan synthétique du projet Novapolis Résilience

Le projet intégrateur **Novapolis Résilience** a permis de concevoir et de développer une solution logicielle complète, modulaire et hautement reproductible, répondant rigoureusement aux exigences d'analyse, d'évaluation et d'optimisation des infrastructures critiques de la métropole.

En s'appuyant sur une modélisation mathématique rigoureuse des graphes (graphes orientés pondérés pour le réseau routier et non orientés pour le réseau de fibre optique), le système intègre l'ensemble des modules analytiques attendus :

- Un diagnostic précis de l'accessibilité des sites sensibles par des parcours itératifs BFS et DFS.
- Une robustesse accrue dans le calcul des itinéraires de secours grâce aux algorithmes de Dijkstra, Bellman-Ford et Floyd-Warshall.
- Une gestion formelle des anomalies topologiques et des circuits absorbants.
- Une optimisation rigoureuse des coûts d'infrastructure par l'application de Kruskal et Prim, complétée par l'étude des arbres couvrants multiples et des plans de raccordement en cas de panne.
- Une simulation comparative de scénarios de crise macroscopiques couplée à des indicateurs quantitatifs de résilience.

Grâce à une séparation stricte des responsabilités entre la couche algorithmique, les services de résilience, l'API REST FastAPI, l'interface visuelle et l'outil en ligne de commande (`cli.py`), l'outil dote les gestionnaires de la métropole d'une capacité d'aide à la décision opérationnelle leur permettant de simuler instantanément l'impact de crises majeures (coupures de routes, pannes de fibre) sur une infrastructure isolée et sécurisée.

6.2 Limites du modèle et perspectives d'évolution

Malgré la robustesse de l'architecture et la rigueur des indicateurs implémentés, l'analyse critique des résultats met en évidence certaines limites inhérentes aux choix de modélisation :

- **Imparfaiteté et staticité des données d'entrée :** Les fichiers CSV et les matrices de coûts reposent sur des estimations nominales ou des états figés qui ne capturent pas la dynamique stochastique instantanée des flux de circulation en temps réel.
- **Hypothèses de linéarisation des coûts :** L'affectation de pondérations fixes (temps ou coûts monétaires) ne prend pas en compte les phénomènes de congestion non linéaire en cas d'afflux massif de véhicules sur un itinéraire de secours secondaire.
- **Persistance et passage à l'échelle (Scalabilité) :** Le stockage sur fichiers plats et la gestion en mémoire volatile (via NetworkX) ne répondent pas aux exigences d'un environnement de production hautement concurrentiel. Toutefois, le recours direct à une base de données orientée graphe clé en main (ou à un moteur distribué) aurait court-circuité l'objectif pédagogique fondamental du projet, ces solutions encapsulant nativement et de manière opaque les algorithmes fondamentaux d'optimisation.

Afin de pallier ces contraintes et d'élever l'outil vers un standard industriel de pointe, les perspectives d'amélioration envisagées pour les futurs travaux incluent :

1. **L'intégration de flux de données en temps réel :** Connexion de l'API à des capteurs urbains dynamiques et des API de trafic pour actualiser les pondérations routières en continu.
2. **L'optimisation dynamique multi-objectifs :** Déploiement d'algorithmes évolutionnaires ou de programmation multicritère pour arbitrer simultanément entre la minimisation des délais d'urgence, la réduction des coûts d'investissement en fibre et la maximisation de la redondance topologique.

3. **L'extension des interfaces d'administration cartographique :** Enrichissement du front-end pour permettre une visualisation géospatiale interactive avancée et un pilotage direct des scénarios de crise par les services de secours de la métropole.
4. **L'industrialisation de la couche de stockage :** Migration vers une architecture de persistance hybride (couplage d'une base relationnelle/géospatiale ou d'un moteur Big Data type Spark GraphFrames) permettant de conserver la maîtrise de l'implémentation algorithmique tout en garantissant la scalabilité sur des réseaux métropolitains massifs.